Computer Architecture Final Project Report

B11901097 許庭愷　　B11901112 卜紹秦

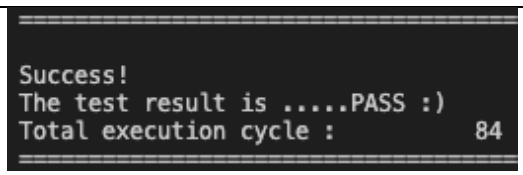
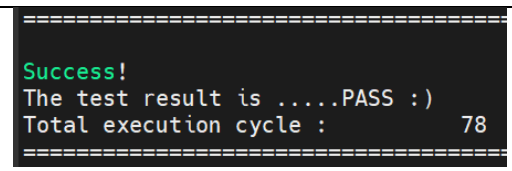
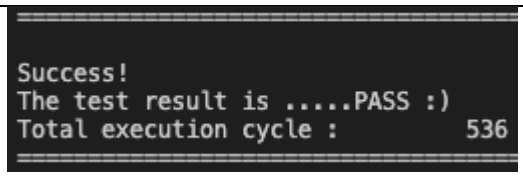
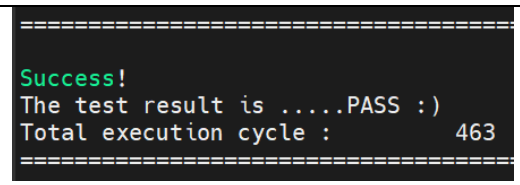
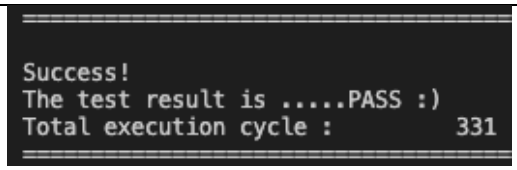
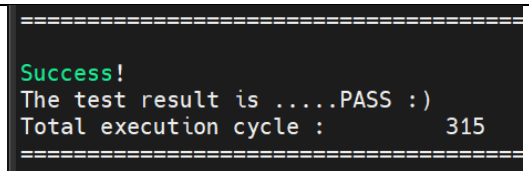
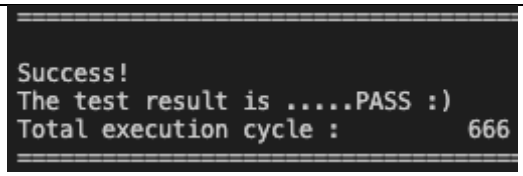
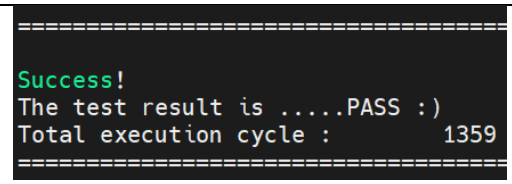
I. Performance

1. Default Test Patterns

Execution cycles:

Test pattern	w. cache (cycles)	w/o cache (cycles)	Speed up
I0	84	78	0.929
I1	536	463	0.864
I2	331	315	0.952
I3	666	1359	2.04

Screenshots:

Test pattern	w. cache (cycles)	w/o cache (cycles)
I0		
I1		
I2		
I3		

2. Modified Test Patterns

Parameters:

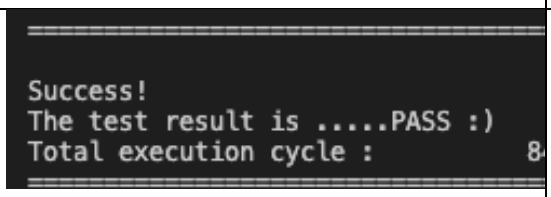
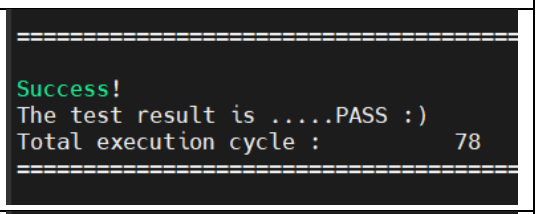
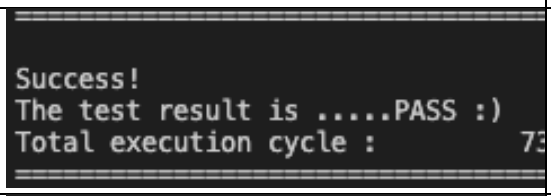
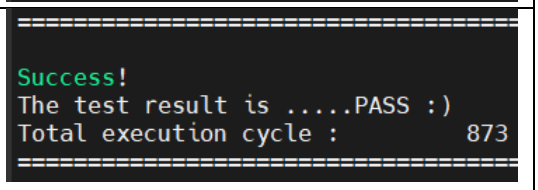
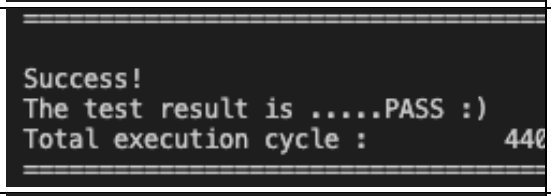
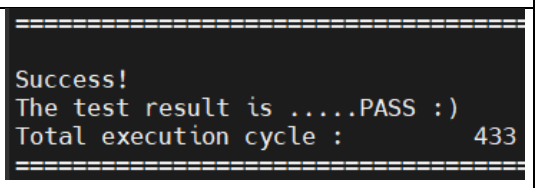
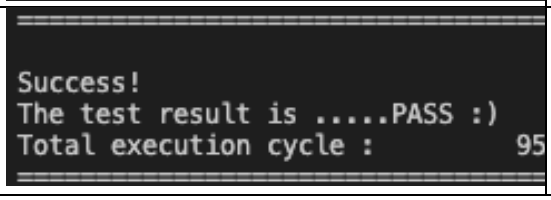
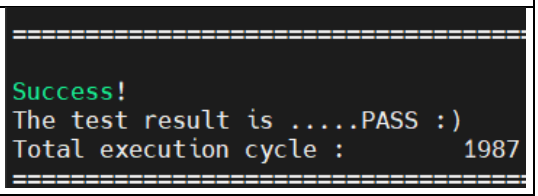
Test Pattern	Parameter
I0	a = 97, b = 25, c = 66, d = 45
I1	n = 10

I2	n = 100
I3	n = 10 v = [9, 7, 3, 1, 5, 8, 0, 9, 10, 2]

Execution cycles:

Test pattern	w. cache (cycles)	w/o cache (cycles)	Speed up
I0	84	78	0.929
I1	736	873	1.19
I2	440	433	0.984
I3	958	1987	2.07

Screenshots:

Test pattern	w. cache (cycles)	w/o cache (cycles)
I0		
I1		
I2		
I3		

II. CPU Architecture

We only use 3 modules that seem to be unable to fulfill the architecture taught in this course, which are CHIP, Reg_file, MULDIV_unit, and it can work! However, our architecture would be too complicated if one would like to modify it into CPU architecture with pipelined structure, so we conclude that this is just a shortcut to realize some simple functions in CPU.

Now let's discuss our CPU architecture and show the difference between the structure discussed in this course.

1. Chip

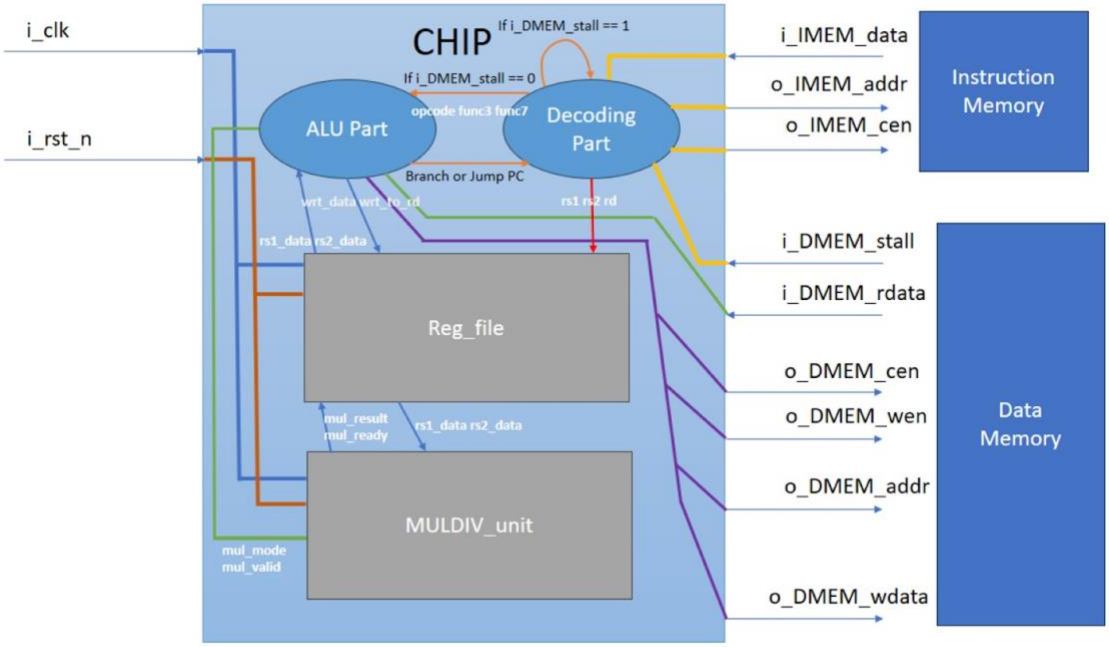


Fig 1. CPU architecture

(here we treat cache as DMEM as if the chip didn't know the existence of cache)

a. Input and Output

i. The Decoding Part

Even though we don't build out additional modules, we can separate the layout of CHIP into the decoding part and the ALU part.

Decoding part would control when to access instruction from instruction memory, which is determined by o_IMEM_cen and o_IMEM_addr.

The action of the decoding part is controlled by i_DMEM_stall, which is activated when the instructions are LW or SW that tells the processor to wait. When i_DMEM_stall is activated, the processor would not go to ALU part, and go to ALU part when deactivated.

The decoding part sends the decoded information to ALU part and Reg_file. It sends opcode, function3, and function7 to ALU part to help it determine which kind of instruction in the current cycle, and sends rs1, rs2, and rd to Reg_file, and the latter further sends the rs1_data and rs2_data to ALU part to operate.

As we've mentioned before, the decoding part determines where to access the instruction from instruction memory, so if the instruction in the current cycle is B-type or J-type, then o_IMEM_addr comes from the ALU part.

ii. The ALU part

The ALU part is told by the decoding part and determines instruction by nested case statement.The primary case statement parameter is opcode, and the secondary case statement parameter is function 3 or function 7.

After determining details of instruction, we directly implement the operation of the corresponding instruction in the case statement. It extracts data of rs1 and rs2 from

Reg_file, and pushes back the data that would be written to rd to Reg_file. If the instruction involves data memory accessing (like LW or SW), the ALU part would give

output signal to control data memory. LW and SW instructions implemented in the ALU part are complicated and involve multiple cycles, which we'd discuss in the later section.

b. Control Signal for One Cycle Instruction

- Instruction Fetch: The decoding part would output control signal to instruction memory and the instruction memory would output the corresponding memory to the decoding part.
- Instruction Decode: The decoding part decodes the instruction memory based on the instruction set listings of RSIC-V, and then directly we operate the instruction in the ALU part.
- Execution: We use several nested case statements to determine the action based on the decoded opcode, function 3, and function 7.
- Memory Access: The data memory access is controlled by the ALU part, and it tells the data memory when to access data memory and when to write data to data memory.
- Write Back: Write back process is also determined by the ALU part, we use wrt_to_rd and wrt_data to control when to write data to rd (like LW and ALU) and what data is written.

c. Combination of Decoding and Operation

As we've shown in part a, we do not implement additional modules in CHIP because we find out that we can directly operate the instruction after knowing it!

However, if one wants to implement a pipelined structure, we should separate the decoding part and the ALU part because we know that they belong to different stages, which are the ID and the EXE part respectively.

2. MULDIV_unit

The MULDIV_unit is activated only when the ALU part receives a signal of multiplication (we don't implement DIV in this CPU), and the ALU part controls the MULDIV_unit when to activate and when to wait for the processor to extract the data memory. The multiplication function is the same as we did in HW2, but we have to implement FSM to control the state of MULDIV_unit, we only need to state multi-cycle operation where we compute multiplication and idle where we do nothing when MULDIV_unit isn't activated.

III. Design of Data Path

We have 2 kinds of instruction that need different numbers of cycles to finish, and we'll show the biggest difference in this section, which is, of course, stall.

1. One cycle instruction:
   This kind of instruction is simple because it isn't involved in accessing data memory or multiplication. The decoding part of CHIP would receive 32 bits instruction, which contains everything we want, and the decoding part would decode the instruction based on the RSIC-V

green card. The decoded information (like rs1, rs2, rd) would be sent to Reg_file and the rest would remain in CHIP main code for the ALU part (like opcode, function 3, and so on).
The ALU part would determine what to do based on the decoded information. If the result needs to be written back to register destination (rd), the ALU part would compute the result and then push back to Reg_file. On the other hand, if the instruction is branch or jump instruction, the ALU part computes the result and then pushes back to the decoding part, which would determine where to extract the instruction memory. All these things would be done in one cycle.

2. Multi-cycle instruction:
   The case can be further separated into the multiplication part and the data memory accessing part.
   i. LW and SW
   The data memory sends a stall signal to the decoding part of CHIP, telling the decoding part to wait (next_PC = PC) and would not do the ALU part, but still decode the instruction and determine the address of the data memory. When i_DMEM_stall is deactivated, the ALU part starts operation (write back to rd for LW and write to the memory for SW). The operation would also be determined by an additional factor, the stall counter, which counts the stalls (in other words, cycles that the processor should wait). The stall counter increases when the opcode is SW or LW and i_DMEM_stall is activated, and it returns to 0 otherwise. This helps the ALU part to determine the moment to access the data memory. When the stall counter is zero and the opcode is LW or SW, this means we are no longer in stall and the processor is able to access to the data memory, otherwise the enable signal (dmem_cen and dmem_wen) should be kept low when we're still in stall.

   ii. Multiplication
   We're forbidden to use the '*' verilog syntax in this CPU and is instructed to use the multiplier implemented in HW2, which requires multiple cycles to finish the instruction. First we assign the inputs of MULDIV_unit of rs1 and rs2, and then we have mul_valid to activate MULDIV_unit and then use mul_ready to control the output of the unit.
   If the mul_ready is high, it means the multiplication is done and we should activate wrt_to_rd to write the result to rd and deactivate the unit. Otherwise, we hold the process (next_PC = PC) until the multiplication is done. In the multiplication unit, because the size of the inputs (operand_a and operand_b) are 32 bits, we expect the total operation cycles would be 32, so we set up a counter to count the cycle. In each cycle, we check if the current bit of operand_b is 1 or 0, if 1 then we shift the operand_a with proper length and add it to the temporary result; if 0, then we keep the temporary result the same. as in the previous cycle. It is useful to regard the multi-cycle operation as for loop in C++ or Python, and we can view the counter as an iterator, so the current bit of operand_b can be expressed as operand_b[cnt-1].
   When the multiplication is done, we pull the ready signal high and put the temporary result to the alu_out, which is assigned to the rd_data.

   For JAL, JALR, AUIPC instructions that aren't mentioned in this course, we merely use the similar method like one-cycle instructions to deal with them. The rough data flow would be, first the decoding part of CHIP receives the instruction memory, and then decode it and sends to the ALU part, which would directly act according to what instruction it be given.
   1. AUIPC, it is simple, we decode the instruction and then write back to rd with data PC + immd.
   2. JAL, as we know, PC would jump to computed address based on PC relative addressing, and we also need to store the PC + 4 to rd, that is, write_data would be PC + 4.
   3. JALR is similar to JAL, only the immediate is different, and we must notice that the immediate should have sign extension.

IV. CPU Layout

```
=============================================================================
|    Register Name    |    Type    | Width | Bus | MB | AR | AS | SR | SS | ST |
=============================================================================
|       rdy_reg       | Flip-flop |   1   |  N  | N  | N  | N  | N  | N  | N  |
|    mode_nxt_reg     | Flip-flop |   2   |  Y  | N  | N  | N  | N  | N  | N  |
|      state_reg      | Flip-flop |   2   |  Y  | N  | N  | N  | N  | N  | N  |
|       cnt_reg       | Flip-flop |   6   |  Y  | N  | N  | N  | N  | N  | N  |
|    mode_now_reg     | Flip-flop |   2   |  Y  | N  | N  | N  | N  | N  | N  |
|       tmp_reg       | Flip-flop |  64   |  Y  | N  | N  | N  | N  | N  | N  |
=============================================================================
                  '/home/raid7_2/userb11/b11097/final_project-2/final_project/01_R
=============================================================================
|    Register Name    |    Type    | Width | Bus | MB | AR | AS | SR | SS | ST |
=============================================================================
|     mem_cen_reg     | Flip-flop |   1   |  N  | N  | Y  | N  | N  | N  | N  |
|     mem_wen_reg     | Flip-flop |   1   |  N  | N  | Y  | N  | N  | N  | N  |
|       PC_reg        | Flip-flop |  31   |  Y  | N  | Y  | N  | N  | N  | N  |
|       PC_reg        | Flip-flop |   1   |  N  | N  | N  | Y  | N  | N  | N  |
=============================================================================

Inferred memory devices in process
        in routine CHIP line 162 in file
                '/home/raid7_2/userb11/b11097/final_project-2/final_project/01_R
=============================================================================
|    Register Name    |    Type    | Width | Bus | MB | AR | AS | SR | SS | ST |
=============================================================================
|  stall_counter_reg  | Flip-flop |  32   |  Y  | N  | N  | N  | N  | N  | N  |
=============================================================================
Warning:  /home/raid7_2/userb11/b11097/final_project-2/final_project/01_RTL/CHIP.
18)
Warning:  /home/raid7_2/userb11/b11097/final_project-2/final_project/01_RTL/CHIP.
18)

Inferred memory devices in process
        in routine Reg_file line 719 in file
                '/home/raid7_2/userb11/b11097/final_project-2/final_project/01_R
=============================================================================
|    Register Name    |    Type    | Width | Bus | MB | AR | AS | SR | SS | ST |
=============================================================================
|       mem_reg       | Flip-flop |  995  |  Y  | N  | Y  | N  | N  | N  | N  |
|       mem_reg       | Flip-flop |  29   |  Y  | N  | N  | Y  | N  | N  | N  |
=============================================================================
```

V. Cache Architecture

This design adopts a write-back, 4-way associative cache with 16 blocks.

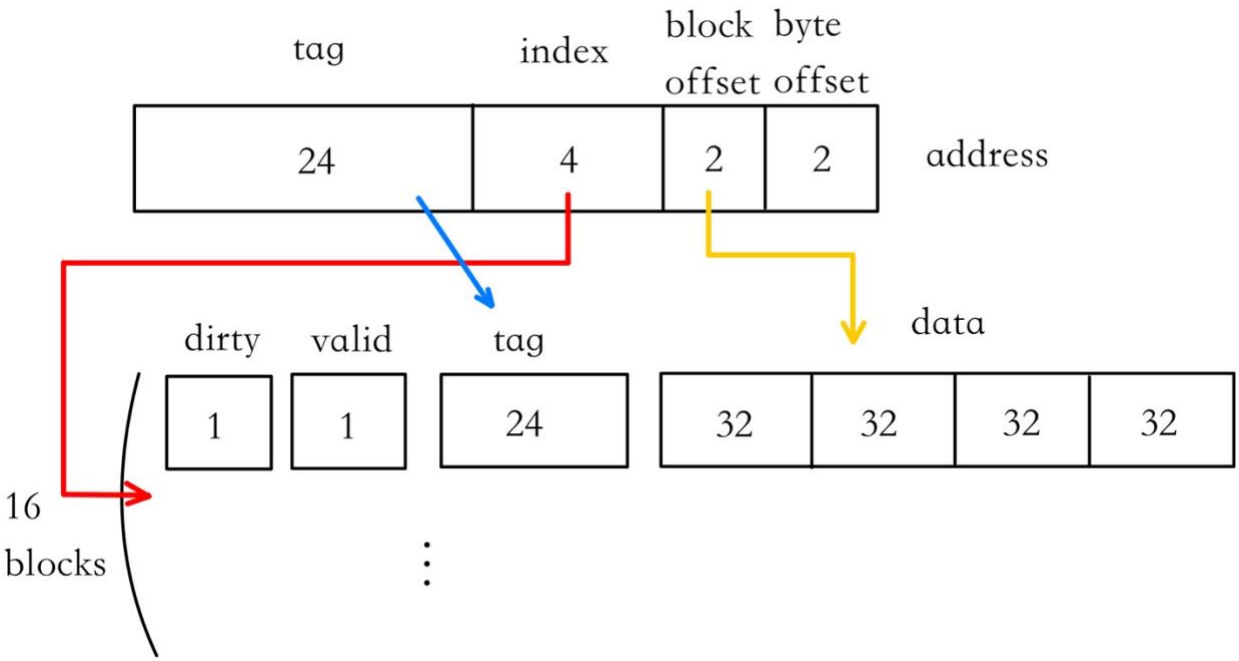


Fig. 2 Cache Structure

1. FSM
   This finite state machine is modified from the one introduced in class with an extra S_FINISH state.

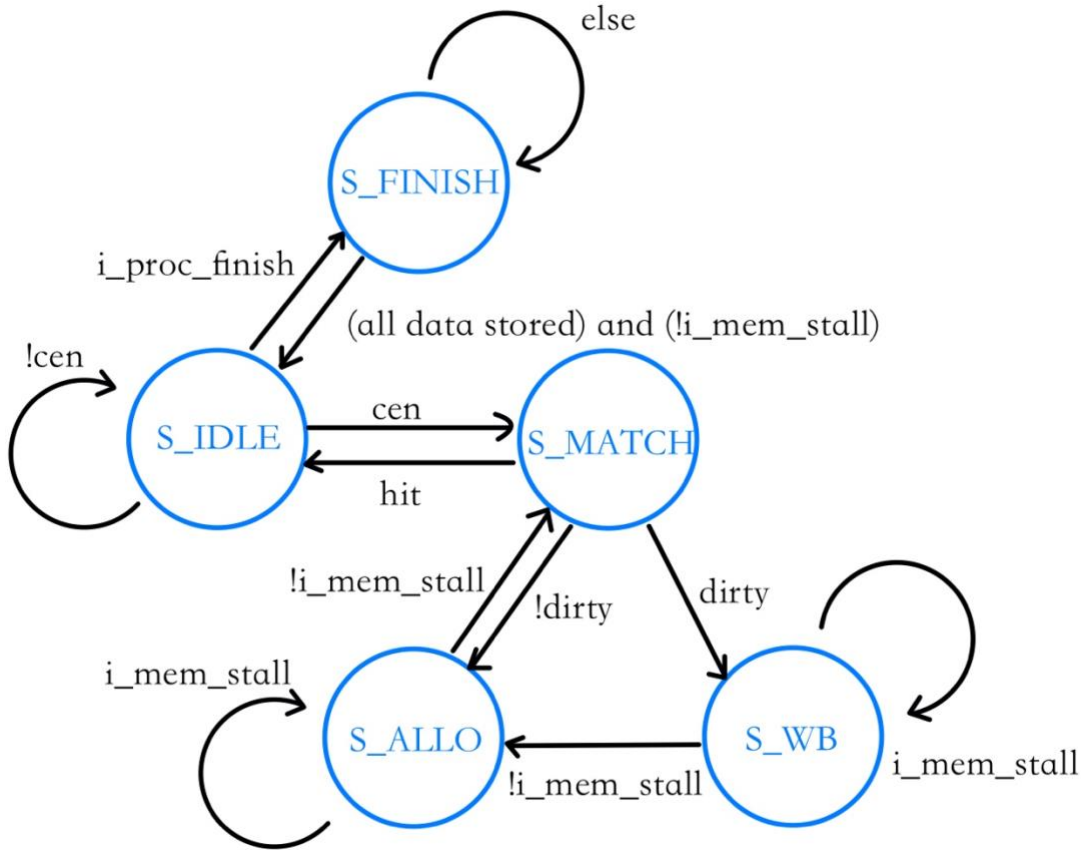


Fig. 3 FSM

State explanation:

State	Action
S_IDLE	Wait for state transition
S_MATCH	Check the address received is a hit or a miss. If it's a hit, return read data or write the data into cache and mark dirty. If it's a miss and the target is dirty, do write back first, load the data from memory, mark valid, then do data processing at S_MATCH; otherwise, skip write back and do similar procedure.
S_WB	Write the dirty data to be replaced back to main memory.
S_ALLO	Load the requested data from main memory to load, set valid to 1 and dirty to 0.
S_FINISH	Ensure all data is written back so that the main memory has correct data values, then set o_cache_finish to high to end the program. Here a counter is utilized to check each block has been written back.

2. Initialization
   At the beginning of the program or with a reset signal, the valid, dirty and data regs are set to zero.
3. Address conversion
   Since a memory block is of 128 bits, while a word is of 32 bits, when accessing the main memory, the address is converted to be byte-addressed instead of word-addressed. That is, we set the least 2 significant bits to 2'b00. Also, the block offset is set to 0 as no particular word in a block is selected when accessing main memory. The address conversion is explained in detail for each state as follows:
   A. S_WB
   ```
   proc_addr = {tag[index], index, offset, 2'b00}
   ```
   At write-back state, we are writing particular block back to the memory, and since we adopt a 4-way associative cache, the index will be the index of the address. The block offset and byte offset are 0 as explained as above.
   B. S_FINISH
   At finish state, we are storing each block back to main memory, which is similar to S_WB except that now we are writing each block back. Thus, the index will be set to the value of a counter. As we previously mentioned, the counter will go from 0~15, therefore number 0~15 block will then be stored.

   C. S_IDLE, S_MATCH, S_ALLO
   For other states, since we are not accessing the main memory, an address conversion is not needed. The address is simply used to access specific data in the cache.

VI. Cache Layout

Register Name	Type	Width	Bus	MB	AR	AS	SR	SS	ST
state_reg	Flip-flop	3	Y	N	Y	N	N	N	N
cen_reg	Flip-flop	1	N	N	Y	N	N	N	N
wen_reg	Flip-flop	1	N	N	Y	N	N	N	N
dirty_reg	Flip-flop	16	Y	N	Y	N	N	N	N
data_reg	Flip-flop	2048	Y	N	Y	N	N	N	N
tag_reg	Flip-flop	384	Y	N	Y	N	N	N	N
valid_reg	Flip-flop	16	Y	N	Y	N	N	N	N

$$\text{register number} = 3 + 1 + 1 + 16 + 2048 + 384 + 16 = 2469 \leq 3000$$

VII. Observation

- CPU part: I find out the jump or branch instruction, if we directly use immediate to compute the following counter address, we would get wrong result. This is because we've used '$signed()' syntax in the implement, which generates wrong result of immediate. For example, if we have the following:

```
JAL: begin
    wrt_to_rd = 1;
    immd = {instr[BIT_W-1], instr[19:12], instr[20], instr[30:21], 1'b0};
    $display("signed(immd):  %h",$signed(immd));
    $display("signed(instr): %h", $signed({instr[BIT_W-1], instr[19:12], instr[20], instr[30:21], 1'b0}));
    $display("nextPC with immd:  %h", $signed(PC) + $signed(immd));
    $display("nextPC with instr: %h", $signed(PC) + $signed({instr[BIT_W-1], instr[19:12], instr[20], instr[30:21], 1'b0}));
```

Take I0 for instance, we would get the following output:

```
signed(immd):  001fffbc
signed(instr): 1fffbc
nextPC with immd:  00210008
nextPC with instr: 00010008
signed(immd):  001fffbc
signed(instr): 1fffbc
nextPC with immd:  00210008
nextPC with instr: 00010008
signed(immd):  001fffbc
signed(instr): 1fffbc
nextPC with immd:  00210008
nextPC with instr: 00010008
signed(immd):  001fffbc
signed(instr): 1fffbc
nextPC with immd:  0020ffc4
nextPC with instr: 0000ffc4
```

Because the size of immediate is not matched with {instr[BIT_W-1], instr[19:12], instr[20], instr[30:21], 1'b0}, we would leave the upper bits of immediate 0, which implicitly suggest the immediate be positive when we use '$signed()' syntax. This would further cause the wrong result or even make the program unstoppable (because we wouldn't never get ECALL instruction).

The solution is to carry out sign extension by hand, given as below.

```
immd[BIT_W-1:21] = (instr[BIT_W-1])? 11'b11111111111:0;//sign extension
```

Therefore, we eliminate the undetermined zero in the upper bits of the immediate.

- Speedup of cache: As we can see, the speedup of cache is not always larger than 1, which is surprising. We think that it is because there would be large miss when the instruction set has fewer data memory accession instruction. The solution would be preload some data memory at the beginning of the program.

Contribution:

50%許庭愷：CPU part of CHIP.v and the report part of CPU

50%卜紹秦：Cache part of CHIP.v and the report part of Cache